



# FINAL PROJECT REPORT

2110215 Programming Methodology

## GROUP MEMBERS

|                       |            |
|-----------------------|------------|
| Suvijak Dowrueng      | 6831370221 |
| Apiwat Khruemasathung | 6831375421 |

# Table of Contents

|                                                                                          |           |
|------------------------------------------------------------------------------------------|-----------|
| <b>1. Game Overview &amp; Manual (How to Play)</b>                                       | <b>1</b>  |
| Objective                                                                                | 1         |
| Win/Loss Conditions                                                                      | 1         |
| Core Gameplay Flow                                                                       | 1         |
| UI Components                                                                            | 4         |
| Controls                                                                                 | 6         |
| Placement Rules (from GameManager.canPlaceTower + GameMap.isBuildable)                   | 6         |
| Tower Status & Upgrade System                                                            | 7         |
| Tower Types (Actual Class Stats)                                                         | 7         |
| Enemy Types (Actual Class Stats)                                                         | 7         |
| <b>2. Game Features &amp; Complexity</b>                                                 | <b>8</b>  |
| Custom Graphics Pipeline                                                                 | 8         |
| Audio Integration (SoundManager)                                                         | 8         |
| Scaling Difficulty via Time-Based Waves                                                  | 8         |
| Fixed-Time Step Game Loop                                                                | 8         |
| Procedural Path Generation + Pathfinding                                                 | 8         |
| <b>3. Overall Architecture &amp; Design</b>                                              | <b>9</b>  |
| application package (View/Presentation Layer)                                            | 9         |
| logic package (Core Game Domain)                                                         | 9         |
| Why this separation matters?                                                             | 9         |
| <b>4. OOP Principles Applied</b>                                                         | <b>9</b>  |
| Inheritance                                                                              | 9         |
| Interface                                                                                | 10        |
| Polymorphism                                                                             | 10        |
| Access Modifiers                                                                         | 11        |
| <b>5. JUnit Testing Strategy</b>                                                         | <b>11</b> |
| <b>6. UML Diagram (It's an SVG, so it can zoom infinitely without losing quality.)</b>   | <b>12</b> |
| src/main/java/application                                                                | 12        |
| src/main/java/logic/enemy, src/main/java/logic/tower, and src/main/java/logic/interfaces | 13        |
| src/main/java/logic/map                                                                  | 14        |
| src/main/java/logic/GameManager.java and src/main/java/logic/DamageText.java             | 15        |
| <b>7. Projects Links</b>                                                                 | <b>15</b> |

# Tower Defense Project (JavaFX)

## 1. Game Overview & Manual (How to Play)

### Objective

Defend your castle for 3 minutes while enemies continuously spawn and move along the generated path. You win if the timer reaches 0 and no enemies remain alive; you lose if castle HP reaches 0.

#### Initial resources:

- Base Health: 100
- Money: 500
- Timer: 03:00

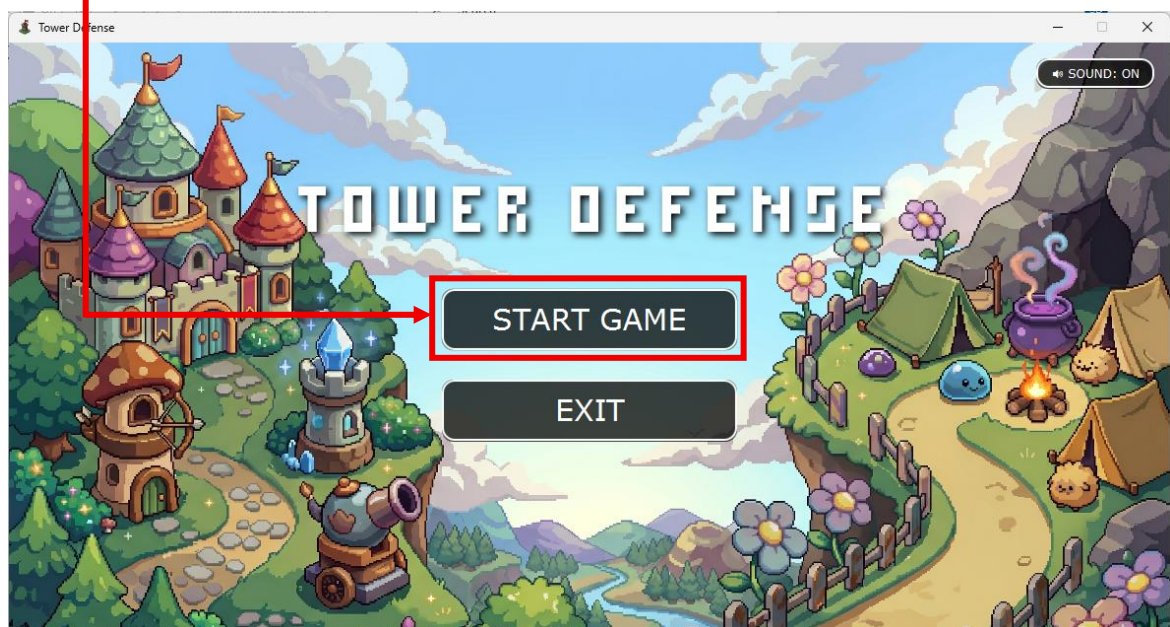


### Win/Loss Conditions

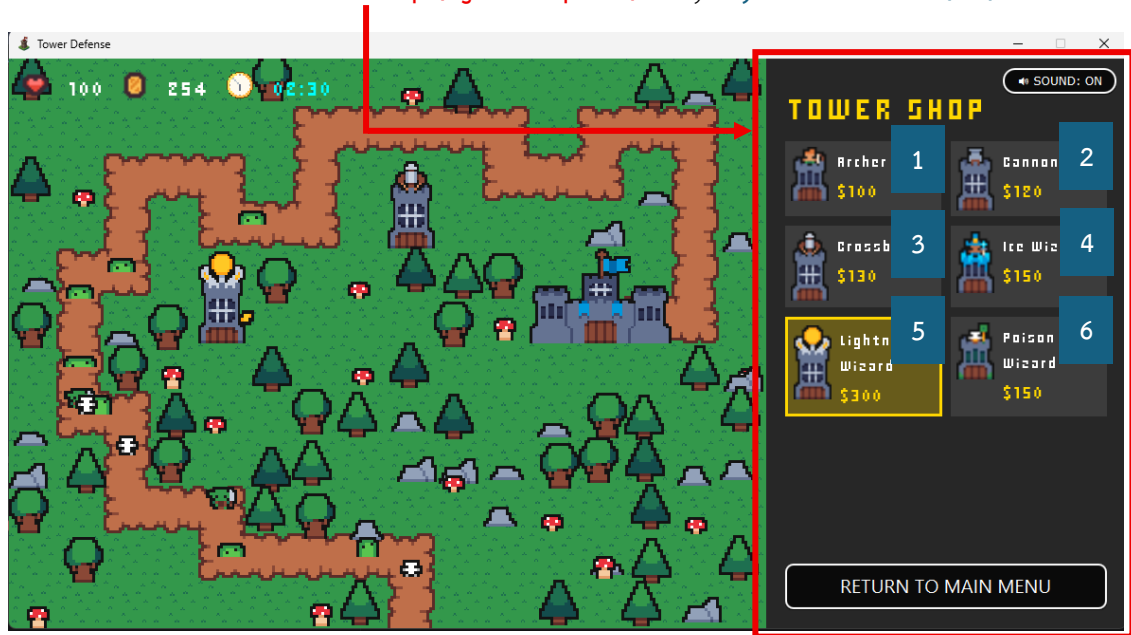
- **Loss:** in GameManager.handleEnemyReachedBase, enemy reaching end reduces base HP; when  $HP \leq 0$ , `isGameOver = true`.
- **Win:** in GameManager.checkAndHandleVictory, when timer expired, no active enemies remain, and base still alive, `isVictory = true`.

### Core Gameplay Flow

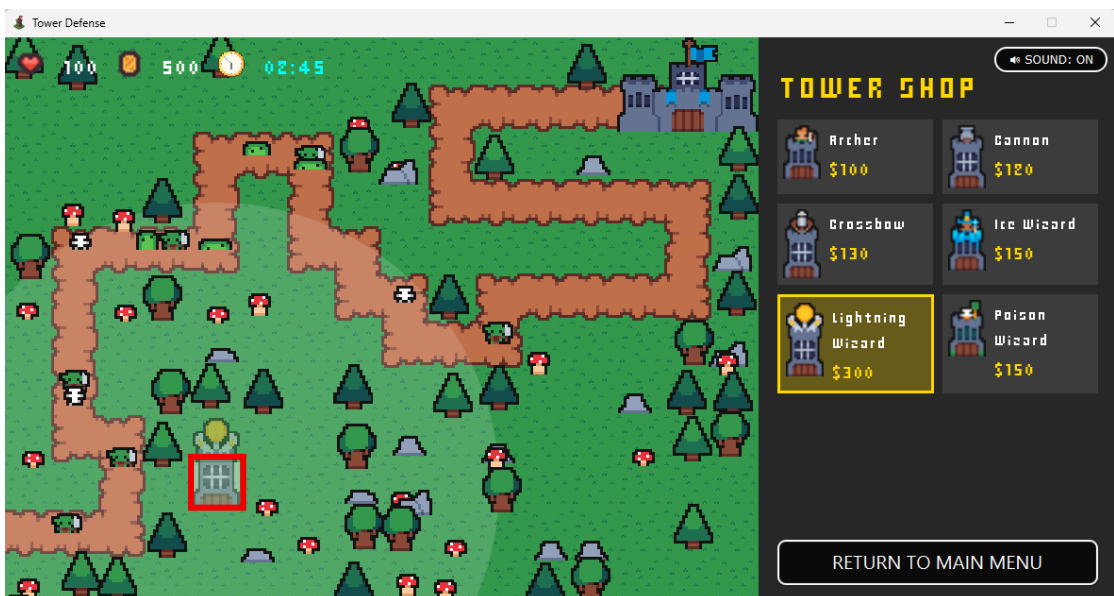
1. **Start** from the main menu (MainMenu) and enter gameplay (`Main.startGameFromMenu()`).



2. Select a tower from the **Tower Shop** (right-side panel) or by **keyboard shortcut (1-6)**.



3. **Place towers** on valid grass tiles (not path, not castle zone, not occupied by other towers, not blocked by decoration).



- Towers **auto-attack** enemies in range.



- Earn money from enemy kills.
- Click a placed tower to open **Tower Status**, then **upgrade** if affordable.



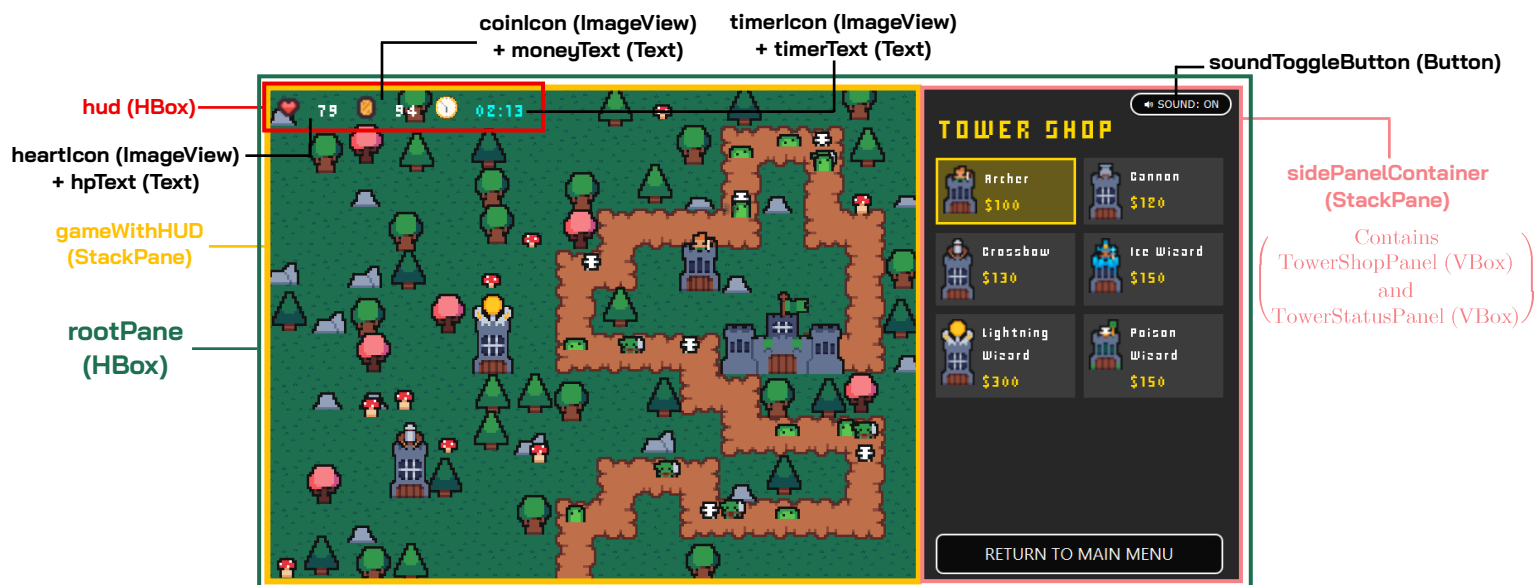
- Survive until time ends.

## UI Components

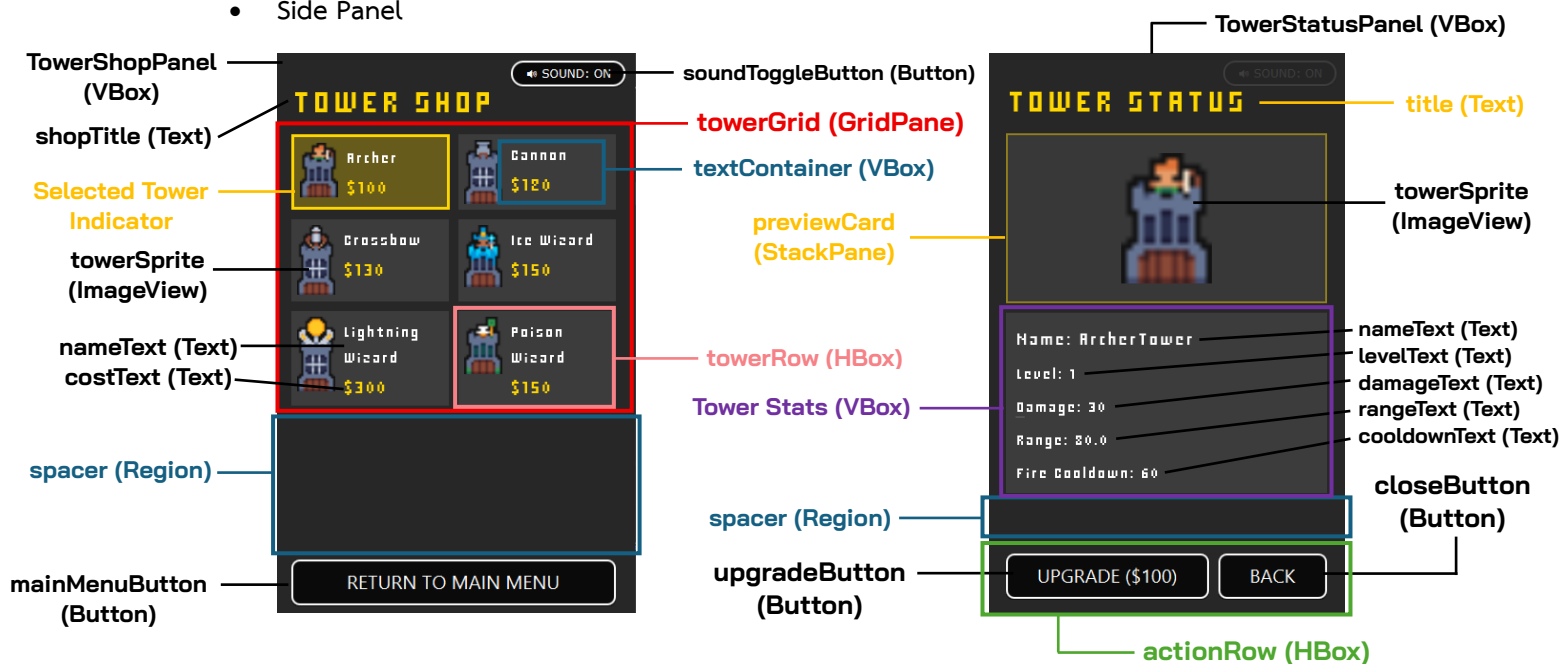
- Main Menu (MainMenu)



- In Game (Main)



- Side Panel



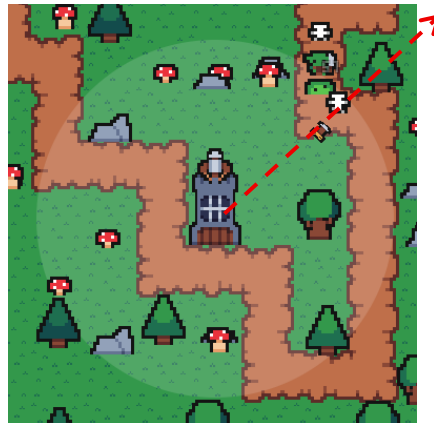


- In-game Visual Feedback

- Hover validity tile highlight & Tower range circles & Ghost tower preview



- Projectile rotation toward targets

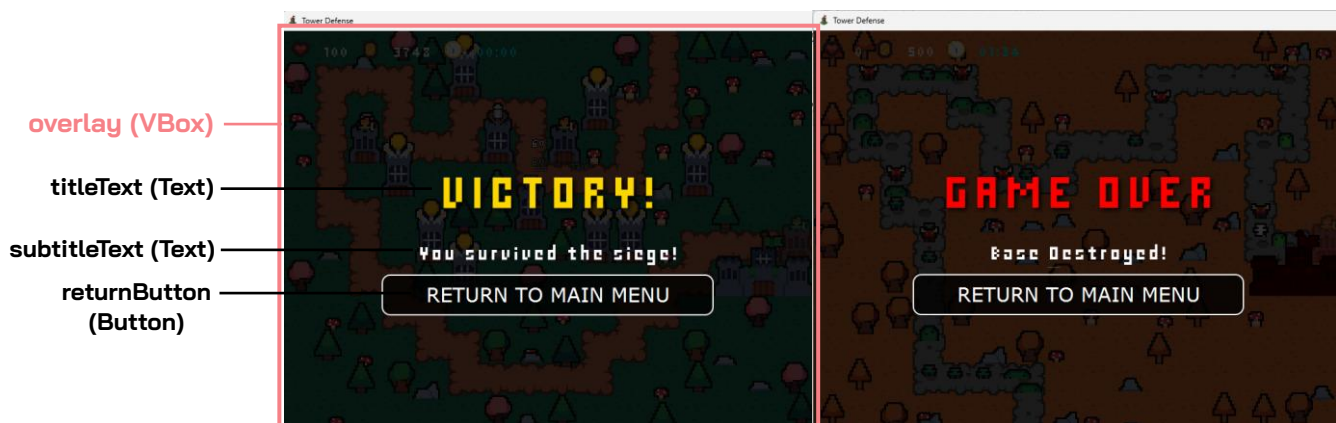


- Enemy hit flash & Floating damage numbers (DamageText) & Castle hit shake effect



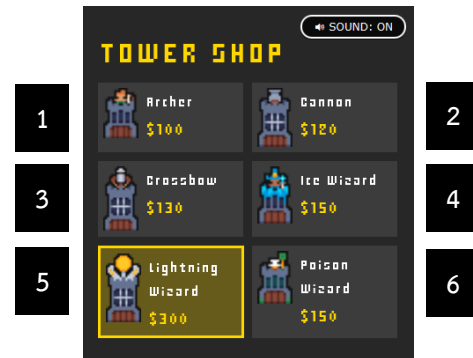
- Terminal Overlays

- Victory overlay & Game Over overlay



## Controls

- **Left Click (Map)**
  - If clicking an existing tower: select/deselect tower and show range.
  - Otherwise: place currently selected tower.
- **Right Click (Map/Shop)**
  - clear selection/cancel placement mode.
- **Keyboard shortcuts (GameView.handleKeyPress)**
  - 1: Archer
  - 2: Cannon
  - 3: Crossbow
  - 4: Ice Wizard
  - 5: Lightning Wizard
  - 6: Poison Wizard



## Placement Rules (from GameManager.canPlaceTower + GameMap.isBuildable)

A tower can be placed only when all conditions are true

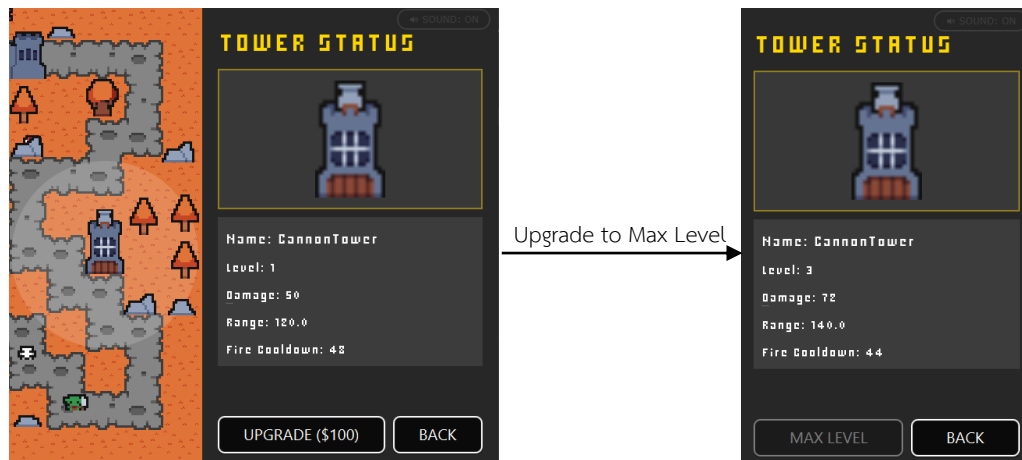
- A tower type is selected.
- Game is not over.
- Tile is inside grid.
- Tile value is buildable grass (0), not path (1) or castle (2).
- Tile is not inside castle clearance zone.
- Tile has no decoration (🌲🌳🏠🏡🏢🏣🏤🏥🏦🏧🏨🏩🏪🏫🏬🏭🏮🏰🏱🏲🏳🏴🏵🏶🏷🏸🏹🏺🏻🏼🏽🏾🏿🏠🏡🏢🏣🏤🏥🏦🏧🏨🏩🏪🏫🏬🏭🏮🏰🏱🏲🏳🏴🏵🏶🏷🏸🏹🏺🏻🏼🏽🏾🏿).
- Tile is not already occupied by another tower.
- Player has enough money for selected tower.





## Tower Status & Upgrade System

Selecting a placed tower opens the **Tower Status Panel** (Main.createTowerStatusPanel) showing



Upgrade behavior (Tower.upgrade())

- level +1 (up to max level 3)
- Damage increases (scaled)
- Range +10
- Fire cooldown decreases (minimum-limited)
- Next upgrade cost increases

## Tower Types (Actual Class Stats)

| Tower Class          | Cost | Damage | Range | Fire Cooldown |
|----------------------|------|--------|-------|---------------|
| ArcherTower          | 100  | 30     | 80.0  | 60            |
| CannonTower          | 120  | 50     | 120.0 | 48            |
| CrossbowTower        | 130  | 35     | 150.0 | 40            |
| IceWizardTower       | 150  | 60     | 150.0 | 120           |
| PoisonWizardTower    | 150  | 60     | 150.0 | 120           |
| LightningWizardTower | 300  | 60     | 300.0 | 60            |

## Enemy Types (Actual Class Stats)

| Enemy Class    | HP  | Speed | Reward | Base Damage to Castle |
|----------------|-----|-------|--------|-----------------------|
| SlimeEnemy     | 100 | 1.0   | 10     | 1                     |
| BatEnemy       | 50  | 2.5   | 15     | 1                     |
| BigSlimeEnemy  | 180 | 0.8   | 20     | 2                     |
| GoblinEnemy    | 90  | 1.8   | 18     | 2                     |
| SkeletonEnemy  | 130 | 1.2   | 16     | 3                     |
| ZombieEnemy    | 170 | 0.9   | 22     | 4                     |
| DemonEnemy     | 250 | 1.5   | 35     | 5                     |
| KingSlimeEnemy | 320 | 2.0   | 45     | 50                    |

## 2. Game Features & Complexity

### Custom Graphics Pipeline

- Centralized texture preloading with AssetManager (environment, towers, projectiles, enemies).
- Theme-dependent map visuals (Theme.NORMAL, Theme.AUTUMN, Theme.SPRING).
- Depth-sorted rendering of decorations/enemies/towers for proper visual layering.

### Audio Integration (SoundManager)

- Dedicated SFX for clicks, enemy hit, castle hit, victory, defeat.
- Separate menu and in-game looping BGM using MediaPlayer.
- Global mute toggle with UI synchronization.

### Scaling Difficulty via Time-Based Waves

- GameManager changes enemy pools and spawn intervals by match phase
  - Early phase: slower spawn interval, easier enemies.
  - Mid phase: medium interval, stronger enemies.
  - Late phase: fastest interval, strongest enemies.

### Fixed-Time Step Game Loop

- Main.createGameLoop() uses an accumulator with fixed logic step
  - LOGIC\_TICKS\_PER\_SECOND = 60
  - LOGIC\_STEP\_SECONDS = 1.0 / 60
  - bounded MAX\_LOGIC\_STEPS\_PER\_FRAME
- important for stable simulation, deterministic update rhythm, reduced frame-rate dependency.

### Procedural Path Generation + Pathfinding

- PathGenerator.generateRandomPath() creates randomized valid maps.
- GameMap.generatePath() reconstructs route to castle via BFS.
- Combined with tower-placement constraints, this increases replayability and algorithmic depth.

### 3. Overall Architecture & Design

#### application package (View/Presentation Layer)

- JavaFX bootstrapping and scene management (Main, Launcher, MainMenu)
- Rendering and player input (GameView)
- UI helpers and assets/audio (UIUtils, AssetManager, SoundManager)

#### logic package (Core Game Domain)

- Match state orchestration (GameManager)
- Combat entities (logic.tower.\*, logic.enemy.\*)
- Map/path systems (logic.map.\*)
- Interfaces/contracts (logic.interfaces.\*)

#### Why this separation matters?

- Easier maintenance and debugging (UI issues vs gameplay issues are isolated).
- Better testability (core mechanics are unit-tested without full JavaFX rendering).
- Cleaner scaling for future features (new UI widgets or new enemy/tower classes can evolve independently).

### 4. OOP Principles Applied

#### Inheritance

- Tower is an abstract base class concrete towers extend it
  - ArcherTower, CannonTower, CrossbowTower, etc.
- Enemy is an abstract base class enemy variants extend it
  - SlimeEnemy, BatEnemy, DemonEnemy, etc.

#### Code example

```
public abstract class Tower implements Upgradable { ... }
public final class ArcherTower extends Tower { ... }

public abstract class Enemy implements Damageable { ... }
public final class DemonEnemy extends Enemy { ... }
```

- Why it is important?
  - Shared fields and logic (position, stats, cooldowns, movement, health, etc.) are implemented once in base classes.
  - Reduces duplication and keeps balancing changes centralized.
  - New tower/enemy variants can be added quickly with minimal code.

## Interface

- Upgradable interface implemented by Tower
- Damageable interface implemented by Enemy

### Code example

```
public interface Upgradable {  
    void upgrade();  
}  
  
public interface Damageable {  
    void takeDamage(int amount);  
}
```

- Why it is important?
  - Defines behavior contracts independent of concrete class.
  - Encourages consistent APIs for upgrade and damage interactions.
  - Supports future extensibility (other entities can become upgradeable/damageable without changing consumers).

## Polymorphism

- Runtime dispatch through base-class references
  - GameManager stores List<Tower> and List<Enemy>
  - Calls tower.update(...) and enemy.update(...) without needing subtype-specific branching
- Tower subclasses override update(...) to use their own projectile sprite/behavior.

### Code example

```
for (Tower tower : activeTowers) {  
    tower.update(activeEnemies, activeProjectiles);  
}
```

```
@Override  
public void update(List<Enemy> enemies, List<Projectile> activeProjectiles) {  
    updateProjectileAttack(enemies, activeProjectiles, CANNON_PROJECTILE_SPRITE);  
}
```

- Why it is important?
  - Same method call, different runtime behavior by tower type.
  - Keeps the combat loop clean and open for extension.
  - Avoids large if/else or switch blocks for every subtype behavior.

## Access Modifiers

- **private (encapsulation, internal invariants)**
  - e.g., GameManager state fields (playerMoney, baseHealth, timeRemaining, etc.)
  - helper methods hidden from external misuse (isPlacementRequestValid, isTileOccupiedByTower)
- **protected (shared subclass access)**
  - e.g., Tower and Enemy combat/state fields used by subclasses for specialization
- **public (external API)**
  - game-facing operations like update, getters/setters, placeTower, tryUpgradeTower

### Code example

```
// GameManager
private int playerMoney;
public boolean placeTower(int row, int col) { ... }

// Tower
protected int damage;
```

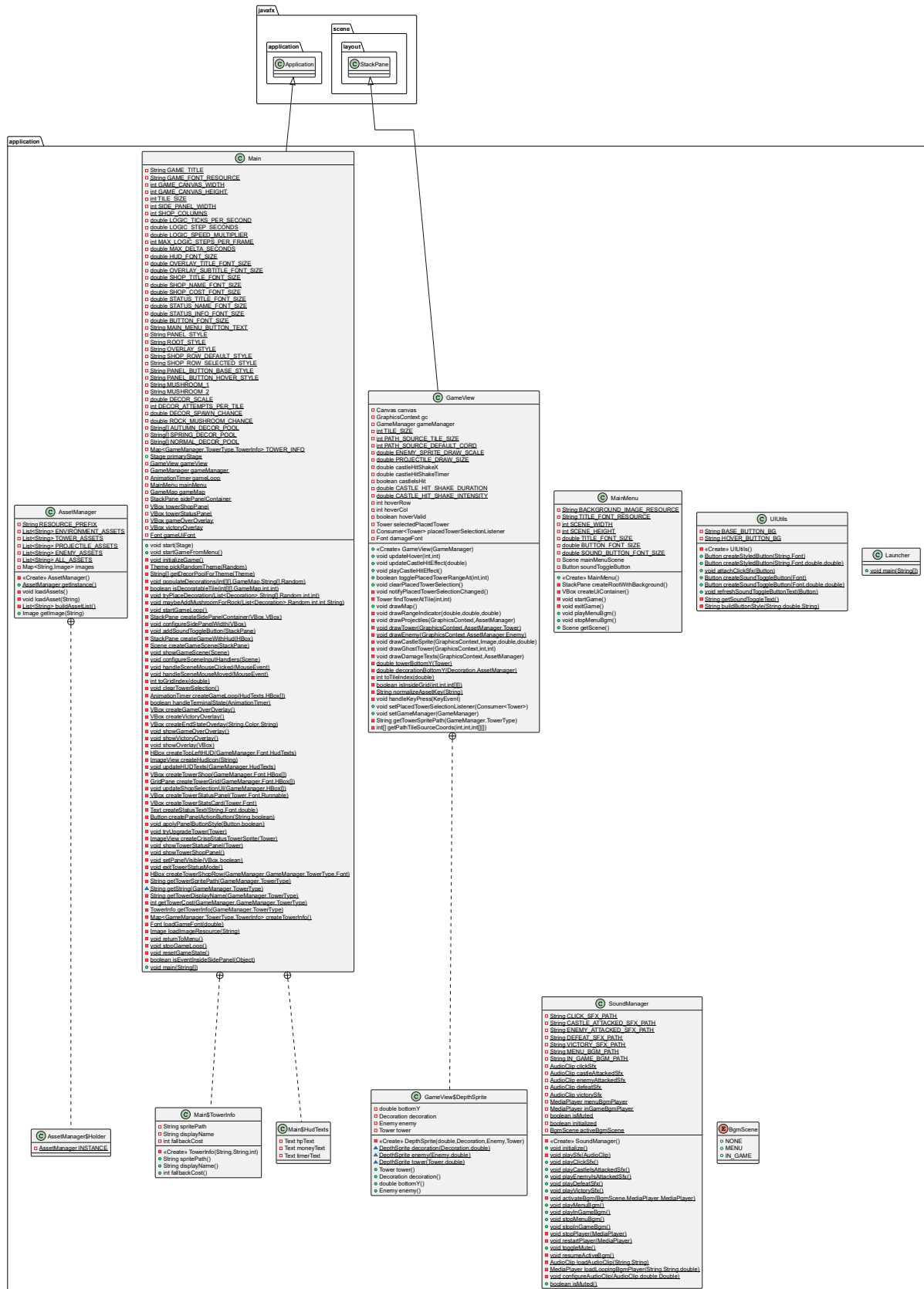
- Why it is important?
  - Protects critical game state from unsafe direct manipulation.
  - Gives controlled extension points to child classes.
  - Exposes only what other modules (UI/tests) must use.

## 5. JUnit Testing Strategy

The src/test/java/logic suite focuses on **core gameplay logic** (high-value, deterministic units), not heavy JavaFX rendering

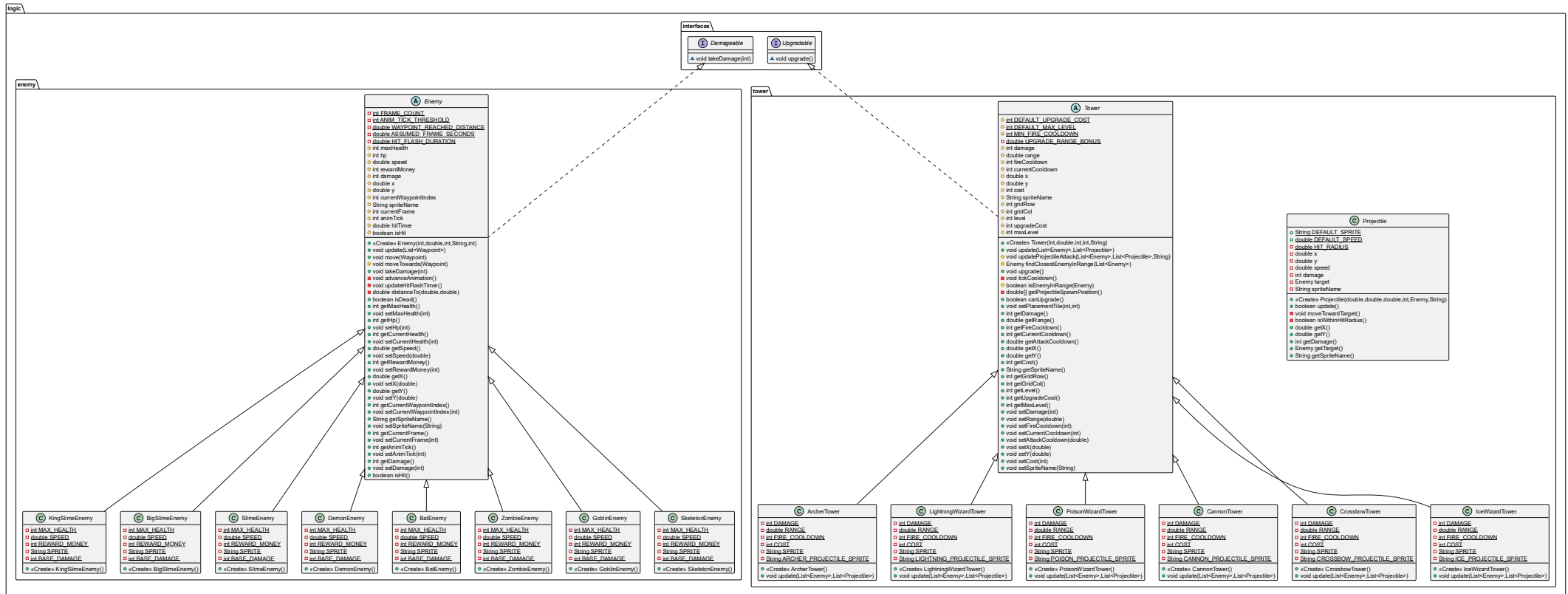
- **TowerTest:** targeting priority, cooldown behavior, upgrade rules, projectile sprite correctness.
- **EnemyTest:** movement/path following, hit flash timing, animation stepping, variant stats.
- **ProjectileTest:** movement, hit radius, overshoot protection, default sprite fallback.
- **GameMapTest:** buildability constraints, waypoint coordinate generation, BFS path behavior.
- **PathGeneratorTest:** generated map validity, single border entry, castle reachability.
- **LevelLoaderTest:** map parsing and fallback behavior on malformed/missing input.
- **DamageTextTest:** fade/lifetime and opacity clamping.
- **GameManagerTest:** economy, placement validity, upgrades, spawn placement, reset flow, victory/game-over transitions.

```
src/main/java/application
```

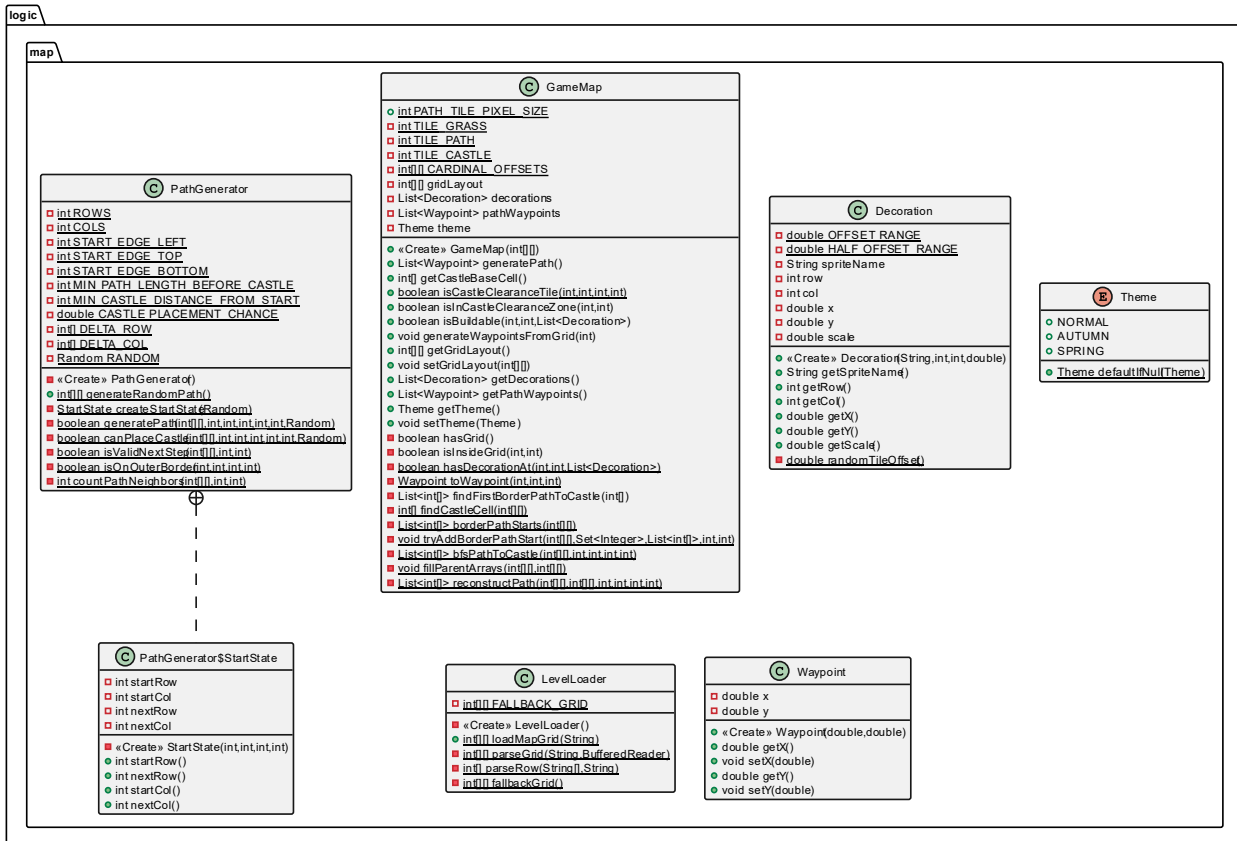




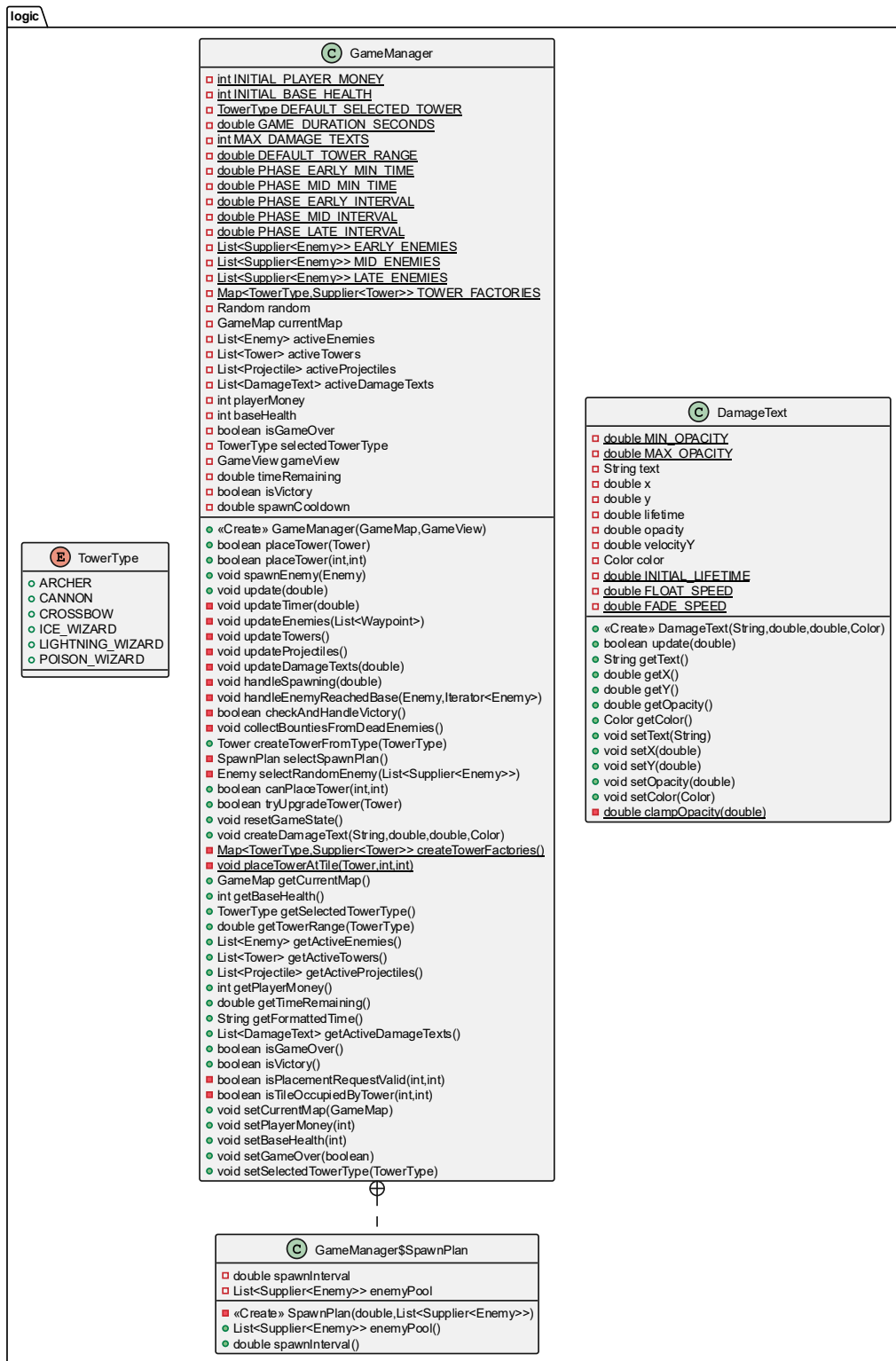
src/main/java/logic/enemy, src/main/java/logic/tower, and src/main/java/logic/interfaces



## src/main/java/logic/map



## src/main/java/logic/GameManager.java and src/main/java/logic/DamageText.java



## 7. Projects Links

- JavaDoc: [Click here to view JavaDoc](#)
- UML Diagram: [Click here to view full UML Diagram](#)